

image-fourier-transform

September 7, 2024

1 Fourier Transform

1.1 Background

All the signals can be written as a sum of sines and cosines. The sum of sines and cosines then can be said to be approximating signal of the original signal. Fourier Transform, gives the Fourier Spectrum which defines the approximating signal. In simpler words, FT allows to extract the information about the components that make up the approximating signal. More precisely Fourier Transform gives information about amplitude and phase. Amplitude gives the information about how much a specific frequency contributes to the construction of approximating signal and phase gives the information of where the frequency components are present. The inverse Fourier Transform is compiling these defining parts of the approximating signal, that is found from Fourier Transform, to construct the actual approximating signal.

For this report signal and function are interchangeable.

2 Continious Fourier Transform and its Inverse in 1D

2.1 Fourier Transform(FT):

Say a non-periodic function $f(t)$, can be written as a sum of sines and cosines. The spectrum that defines this approximating function made up of sum of sines and consines, can be computed by the following Continuous Fourier Transform,

$$F(u) = \int_{-\infty}^{\infty} f(t)e^{-2\pi iut} dt \quad (1)$$

$$F(u) = \int_{-\infty}^{\infty} f(t)(\cos(2\pi ut) - i\sin(2\pi ut))dt \quad (2)$$

$$F(u) = \int_{-\infty}^{\infty} f(t)(\cos(2\pi ut))dt - i \int_{-\infty}^{\infty} f(t)\sin(2\pi ut)dt \quad (3)$$

$$F(u) = \text{Real}(F(u)) - i\text{Imaginary}(F(u)) \quad (4)$$

Here u = frequency, i = imaginary number. From the above equations it is evident that, at each frequency value, a complex output is recieved. From this complex number the amplitude and phase can be computed. It is this ampliitude and phase that defines the underlying signal that approximates the original signal. Amplitude = $|F(u)|$, Phase = $\text{ArcTan}(\text{Imaginary coeff}/\text{Real coeff})$.

Also note that, it can be shown the Fourier Transform is real iff the original signal $f(t)$ is even. But almost always, doesn't matter original signal $f(t)$ is real or not, the Fourier Transform is complex.

In polar form the equivalent of the above equations can be written as,

$$F(u) = A(u)e^{i\phi(u)} \quad (5)$$

Here $A(u)$ = amplitude at frequency u and $\phi(u)$ = phase at frequency u

Additionally, Fourier Transform can be treated also as a tool to take a signal from the time/spatial domain to the frequency domain as the original signal is a variable of t and after FT the signal becomes a variable of u . This point of view of the Fourier Transform is most dominant.

2.2 Inverse Fourier Transform(IFT):

Say the FT of the non-periodic signal is defined, then we know the necessary information about the components that define the underlying approximating signal. Now in Inverse Fourier Transform, these information about the component is used to put together the approximating signal. The continuous IFT is defined as,

$$f(t) = \int_{-\infty}^{\infty} F(u)e^{2\pi iut} du \quad (6)$$

If $f(t)$ is a real signal then the Inverse Fourier Transform will also be real.

2.3 Example: Fourier Transform of $\text{Sin}(2\pi t)$

As a preliminary knowledge for this section, the dirac delta function needs to be defined. The dirac delta also known as the impulse function is defined as the following.

$$\delta(t - t_0) = \begin{cases} 1 & \text{if } t = t_0 \\ 0 & \text{if } t \neq t_0 \end{cases} \quad (7)$$

The delta function is interpreted as, having spike of magnitude 1 at location $t = t_0$. It is 0 everywhere else.

For illustration of the fourier transform, in this report the function $\text{Sin}(2\pi t)$ is used a lot. So it is logical to understand the FT of it.

Say for $\text{Sin}(2\pi t)$, frequency(u) = 1 is considered, then in this section the job is to understand what it means to take the fourier transform of $\text{Sin}(2\pi t)$. Period($T = 2\pi/2\pi u$) and frequency($2\pi u/2\pi$) of the signal is 1. If the signal is known over one period, the signal is known everywhere. So evaluating the Fourier Transform from 0 to 1 should give the necessary spectral information.

$$F(u) = \int_{-\infty}^{\infty} \text{Sin}(2\pi t)e^{-2\pi iut} dt \quad (8)$$

$$F(u) = \int_0^1 \text{Sin}(2\pi t)e^{-2\pi iut} dt \quad (9)$$

From the famous Euler's formula it can be shown,

$$\sin(2\pi t) = \frac{e^{2\pi it} - e^{-2\pi it}}{2i} \quad (10)$$

Plugging the values in the FT equation,

$$F(u) = \frac{1}{2i} \left(\int_0^1 e^{2\pi i(1-u)t} dt - \int_0^1 e^{-2\pi i(u+1)t} dt \right) \quad (11)$$

From proof in appendix the FT equation becomes,

$$F(u) = \frac{1}{2i} \left(\int_0^1 e^{2\pi i(1-u)t} dt - \int_0^1 e^{-2\pi i(u+1)t} dt \right) = \frac{1}{2i} (\delta(u-1) - \delta(u+1)) = \left(\frac{1}{2i} \delta(u-1) - \frac{1}{2i} \delta(u+1) \right) \quad (12)$$

From the FT equation the spectrum is obtained and from the spectrum it is clear that the function $\sin(2\pi t)$ has impulses at -1 and 1 frequencies (by definition of dirac delta function). This means, those are the only points where there would be Amplitude. From the results it can be generalized that, if the considered Sine function has frequency 5, then it would have impulses at -5 and +5 frequencies.

3 Discrete Fourier Transform(DFT) and its inverse(IDFT):

The goal for this report is to use FT on images. Since the images are discrete and not continuous, the Discrete Fourier Transform is required. The DFT is defined as,

$$F(u) = \sum_{x=0}^{M-1} f(x) e^{-2\pi i u x / M} \quad (13)$$

Conventionally the variable x is taken as a discrete variable instead of t . here u = frequency, x = spatial variable, M = Total number of samples in the signal

The transition from Continuous Fourier Transform to Discrete Fourier Transform can be shown through proofs, but they are avoided.

The Inverse Discrete Fourier Transform is defined as,

$$f(x) = \frac{1}{M} \sum_{u=0}^{M-1} F(u) e^{2\pi i u x / M} \quad (14)$$

The existence of DFT means that it is possible to take FT of a bunch of discrete points. Fast Fourier Transform algorithm is used to compute the DFT quickly.

4 1D Fourier Transform

The ultimate goal is to take the DFT of an image to find the spectrum that defines the approximating signal (made of sum of sines and cosines). Image is nothing but array of discrete samples. So I take an array 1D of discrete samples to visualize what might be happening when the DFT of this array is taken.

For this illustration, discrete samples are taken from the $\sin(2\pi \cdot 5x)$ function, which has frequency $(2 \cdot 5\pi)/2\pi = 5$.

It is already said that, FT of a signal having frequency 5, means it will only have amplitude values at -5 and +5 frequencies and 0 elsewhere. So if the Amplitude spectrum is plotted there should be a spike seen at the -5 and +5 frequency points. Now the 1D transform is taken to show that the theoretical results match with the actual results.

```
[1]: # Necessary libraries
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

# Preparing the 1D signal
x = np.arange(0, 1.0, 0.01)
# print(len(x))
freq = 5
y = np.sin(2*np.pi * freq * x)

# Computing the DFT. The DFT will have the same shape as y.
# For faster computation fft is used
fft = np.fft.fft(y)

# Computing the amplitude and phase
amplitude = np.abs(fft)
phase = np.angle(fft)

# Computing the frequencies for spectrum
n = len(y)
frequencies = np.fft.fftfreq(n, 0.01)
# print(frequencies)

# plotting
fig, ax = plt.subplots(2,2)
fig.set_size_inches(12,8)

ax[0,0].plot(x, y)
ax[0,0].set_title("Continuous Sin(2pi5x)")
ax[0,0].set_xlabel('X axis')
ax[0,0].set_ylabel('Y axis')
ax[0,1].scatter(x,y)
ax[0,1].set_title("Sampled Sin(2pi5x)")
```

```

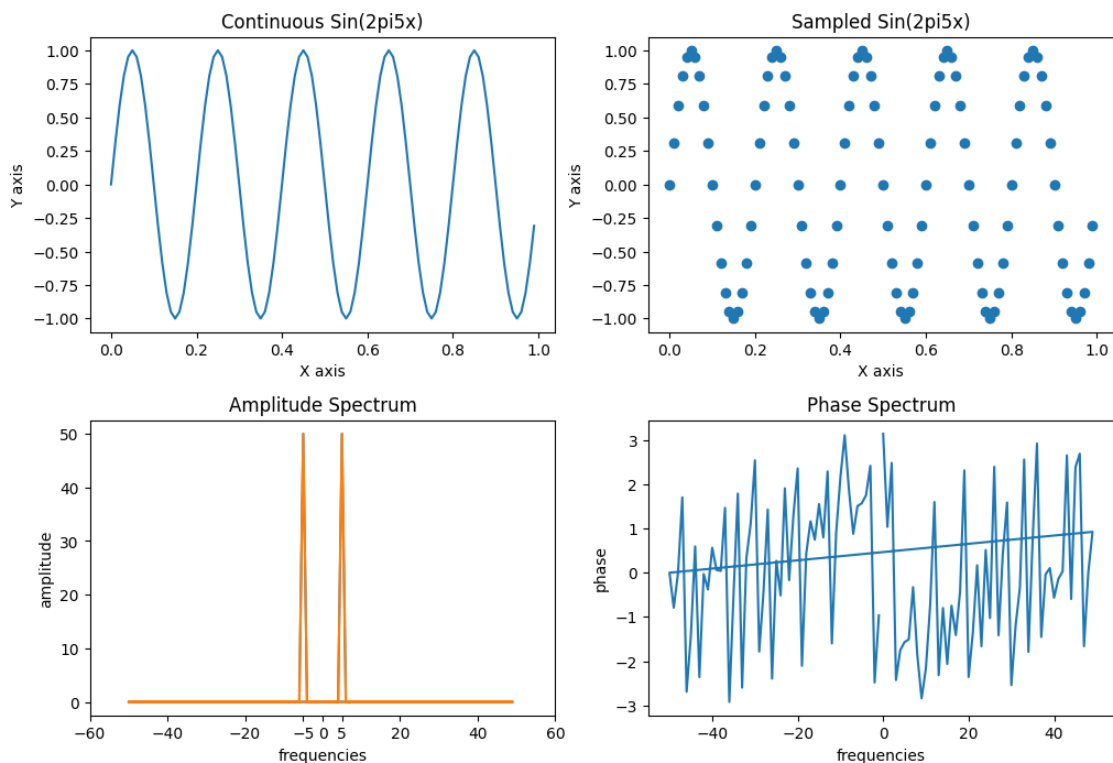
ax[0,1].set_xlabel('X axis')
ax[0,1].set_ylabel('Y axis')
# ax[0,1].plot(x,y)
ax[1,0].plot(frequencies, amplitude)
ax[1,0].set_title("Amplitude Spectrum")
ax[1,0].set_xlabel('frequencies')
ax[1,0].set_ylabel('amplitude')

lines = ax[1,0].plot(frequencies, amplitude)
ax[1,0] = lines[0].axes
ax[1,0].set_xticks(list(ax[1,0].get_xticks()) + [-5, 5])

ax[1,1].plot(frequencies, phase)
ax[1,1].set_title("Phase Spectrum")
ax[1,1].set_xlabel('frequencies')
ax[1,1].set_ylabel('phase')
plt.subplots_adjust(hspace = 0.3)

plt.show()

```



From the amplitude spectrum of the sampled function $\sin(2\pi 5x)$, it is visible that there are two

impulses at -5 and +5 frequencies. This result is consistent with the theoretical results. Another observation in the amplitude spectrum is that the frequency values move from $-M/2$ to $M/2$, where N is the number of samples. This is because $M/2$ is the Nyquist Frequency, which guarantees that, to represent the underlying signal the highest frequency that might be required is $M/2$.

In the results there is also a phase Spectrum. The Phase spectrum is rather difficult to interpret and it appears to be random. But it would be illustrated that it is an incredibly important part of the signal, specially when reconstruction is considered.

5 Same Magnitude but different phase spectrum

For this section two Sine functions with the same magnitude are taken but to the later function a phase shift of $+\pi$ has been added. The purpose is to understand how the amplitude and phase changes in these two cases. The two functions are $\sin(2\pi 5x)$ and $\sin(2\pi 5x + \pi)$.

```
[2]: import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

# Preparing the 1D signal
x = np.arange(0, 1.0, 0.01)
freq = 5
y = np.sin(2*np.pi * freq * x)
# Phase shift +pi has been added to the signal
y_prime = np.sin(2*np.pi * freq * x + np.pi)

# Computing the DFT. The DFT will have the same shape as y.
fft = np.fft.fft(y)
fft_prime = np.fft.fft(y_prime)

# Computing the amplitude and phase
amplitude = np.abs(fft)
phase = np.angle(fft)

amplitude_prime = np.abs(fft_prime)
phase_prime = np.angle(fft_prime)

# Computing the frequencies for spectrum
n = len(y)
frequencies = np.fft.fftfreq(n, 0.01)

# plotting
fig, ax = plt.subplots(2,3)
fig.set_size_inches(12,8)

ax[0,0].plot(x, y)
ax[0,0].set_title("Continuous Sin(2pi5x)")
```

```

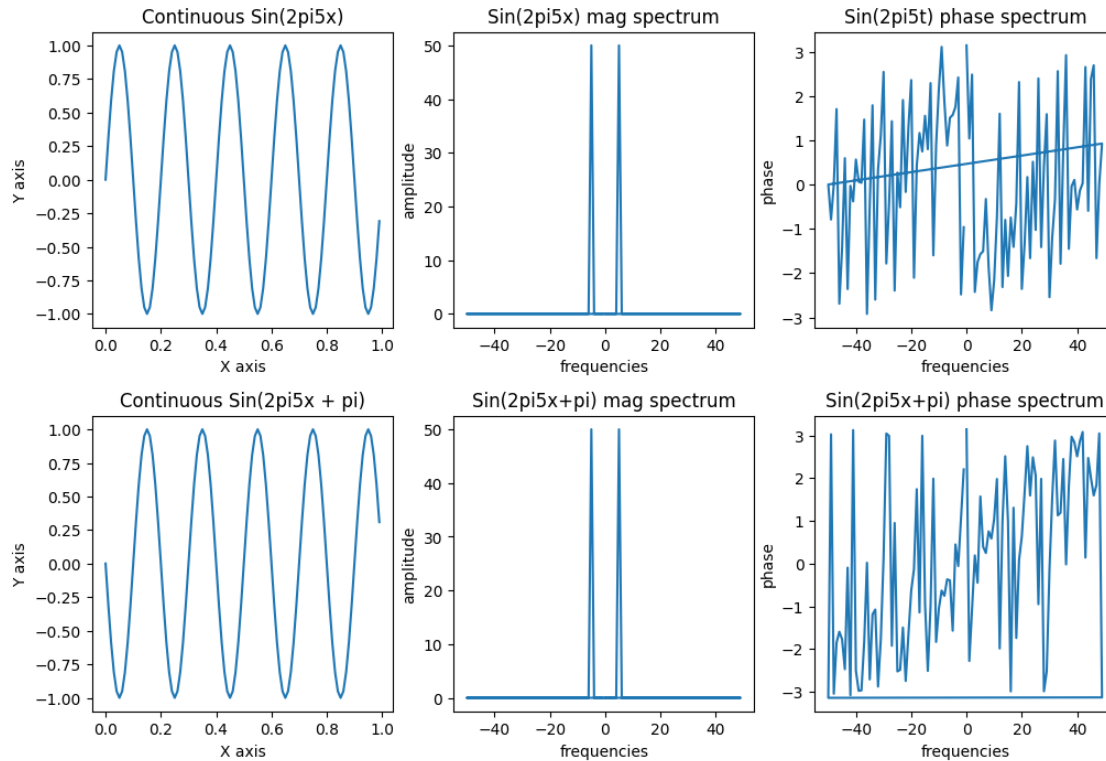
ax[0,0].set_xlabel('X axis')
ax[0,0].set_ylabel('Y axis')
ax[0,1].plot(frequencies, amplitude)
ax[0,1].set_xlabel('frequencies')
ax[0,1].set_ylabel('amplitude')
ax[0,1].set_title("Sin(2pi5x) mag spectrum")
ax[0,2].plot(frequencies, phase)
ax[0,2].set_title("Sin(2pi5t) phase spectrum")
ax[0,2].set_xlabel('frequencies')
ax[0,2].set_ylabel('phase')


ax[1,0].plot(x, y_prime)
ax[1,0].set_title("Continuous Sin(2pi5x + pi)")
ax[1,0].set_xlabel('X axis')
ax[1,0].set_ylabel('Y axis')
ax[1,1].set_xlabel('frequencies')
ax[1,1].set_ylabel('amplitude')
ax[1,1].plot(frequencies, amplitude_prime)
ax[1,1].set_title("Sin(2pi5x+pi) mag spectrum")
ax[1,2].set_xlabel('frequencies')
ax[1,2].set_ylabel('phase')


ax[1,2].plot(frequencies, phase_prime)
ax[1,2].set_title("Sin(2pi5x+pi) phase spectrum")
plt.subplots_adjust(hspace = 0.3)

plt.show()

```



From the above plots, it is seen that the amplitude spectrum of two different functions are same. But there is considerable visual difference between the phase of the functions. So only the amplitude spectrum cannot reliably hold all the information of a signal and phase is also really important.

6 More than one frequencies

For the experiments in this section the signal under consideration is constructed by combining Sine function of different frequencies. The purpose is to analyze what happens when the Discrete Fourier Transform of these sum of sine functions are taken. One function is $\sin(2\pi 5x)$ and the other one is $\sin(4\pi 5x)$

```
[3]: # Preparing the 1D signal
x = np.arange(0, 1.0, 0.01)
freq = 5
y = np.sin(2*np.pi * freq * x) + np.sin(4*2*np.pi * freq * x)

# Computing the DFT. The DFT will have the same shape as y.
# For faster computation fft is used
fft = np.fft.fft(y)

# Computing the amplitude and phase
amplitude = np.abs(fft)
phase = np.angle(fft)
```

```

# Computing the frequencies for spectrum
n = len(y)
frequencies = np.fft.fftfreq(n , 0.01)

# plotting
fig, ax = plt.subplots(2,2)
fig.set_size_inches(12,8)

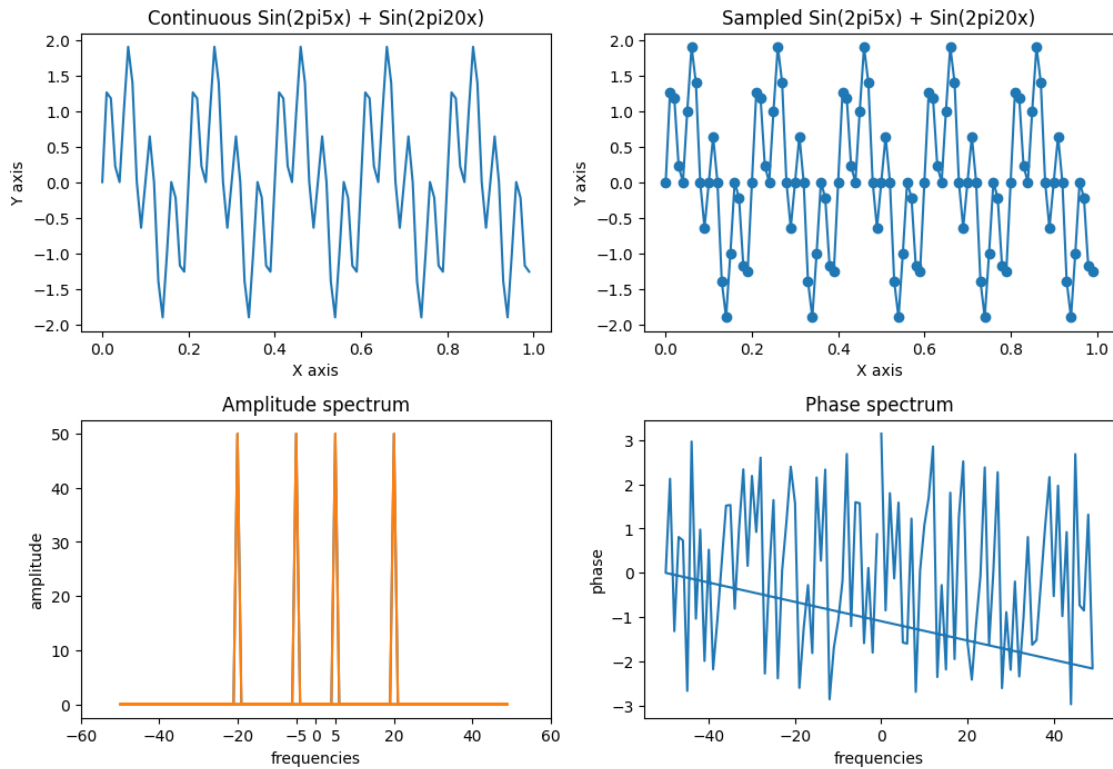
ax[0,0].plot(x, y)
ax[0,0].set_title("Continuous Sin(2pi5x) + Sin(2pi20x)")
ax[0,0].set_xlabel('X axis')
ax[0,0].set_ylabel('Y axis')
ax[0,1].scatter(x,y)
ax[0,1].plot(x,y)
ax[0,1].set_title("Sampled Sin(2pi5x) + Sin(2pi20x)")
ax[0,1].set_xlabel('X axis')
ax[0,1].set_ylabel('Y axis')
ax[1,0].plot(frequencies, amplitude)
ax[1,0].set_xlabel('frequencies')
ax[1,0].set_ylabel('amplitude')
lines = ax[1,0].plot(frequencies, amplitude)
ax[1,0].set_title("Amplitude spectrum")
ax[1,0] = lines[0].axes
ax[1,0].set_xticks(list(ax[1,0].get_xticks()) + [-5, 5,])

ax[1,1].plot(frequencies, phase)
ax[1,1].set_title("Phase spectrum")
ax[1,1].set_xlabel('frequencies')
ax[1,1].set_ylabel('phase')

plt.subplots_adjust(hspace = 0.3)

plt.show()

```



From the amplitude spectrum it is visible that because the signal is the sum of sine functions of frequencies 5 and 20, there are impulses at -5,5 as well as -20,20. The important observation from all these previous experiment is that the fourier transform will give information about which frequencies are used to construct the approximating signal.

It is to be noted that sine graphs at the top have sharp edges because the sampling interval is too large for such rapidly changing signal. In other words the integrity of the signal has been compromised. This is the price paid for a good looking Amplitude spectrum.

7 1D Inverse Fourier Transform

Now that some illustrations have been done on what the 1D FT looks like in different contexts, it is time to take the Inverse Discrete Fourier Transform of the Fourier spectrum to get the original signal. The Signal considered in this example is $\sin(2\pi 5x)$

```
[4]: # Necessary libraries
import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

# Preparing the 1D signal
x = np.arange(0, 1.0, 0.01)
freq = 5
```

```

y = np.sin(2*np.pi * freq * x)

# Computing the DFT. The DFT will have the same shape as y.
# For faster computation fft is used
fft = np.fft.fft(y)

# Computing the amplitude and phase
amplitude = np.abs(fft)
phase = np.angle(fft)

# Computing the frequencies for spectrum
n = len(y)
frequencies = np.fft.fftfreq(n , 0.01)

# Computing the inverse of the fft signal
# Since the signal is real the Inverse of the fourier must be real
inverse_transform = np.real(np.fft.ifft(fft))

# plotting
fig, ax = plt.subplots(2,2)
fig.set_size_inches(12,8)

ax[0,0].scatter(x, y)
ax[0,0].set_xlabel('X axis')
ax[0,0].set_ylabel('Y axis')
ax[0,0].plot(x, y)
ax[0,0].set_title("Sampled Sin(2pi5x)")
ax[0,1].plot(frequencies, amplitude)
ax[0,1].set_ylabel('Amplitude')
ax[0,1].set_xlabel('Frequencies')

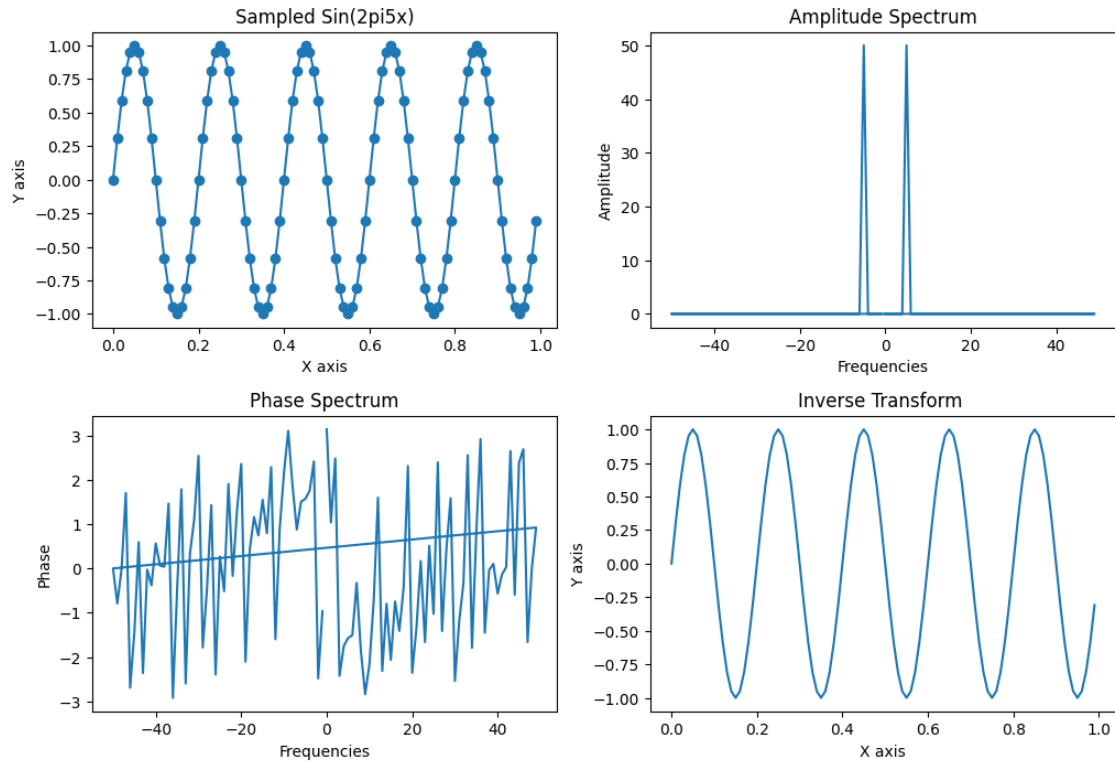
ax[0,1].set_title("Amplitude Spectrum")

ax[1,0].plot(frequencies, phase)
ax[1,0].set_ylabel('Phase')
ax[1,0].set_xlabel('Frequencies')
ax[1,0].set_title("Phase Spectrum")
ax[1,1].plot(x , inverse_transform)
ax[1,1].set_xlabel('X axis')
ax[1,1].set_ylabel('Y axis')
ax[1,1].set_title("Inverse Transform ")

plt.subplots_adjust(hspace = 0.3)

plt.show()

```



7.1 Numerical error complex term

One important insight into Discrete Inverse Fourier Transform is that, if the original signal is real, the Inverse will also be real. Now, for the Inverse to be real the imaginary part of the IDFT needs to be 0. But because everything in DFT and IDFT is computed numerically the imaginary part usually does not equal to 0 but it is very close to 0. The imaginary coefficients of the above IDFT is shown below. There are 100 samples so 100 imaginary coeffs. These coeffs are errors associated with each sample.

Since the imaginary part represents error, only the real part is taken to construct the original signal after IDFT is taken.

```
[5]: print(np.imag(np.fft.ifft(fft)))
```

```
[-4.36918790e-18 -9.46469086e-18  4.10043770e-17  5.25193169e-17
  3.04919906e-17 -7.67858816e-17  4.66598117e-17  4.08189256e-18
  4.20556094e-17 -1.86610729e-16 -2.46084821e-17  1.56504807e-16
 -4.80679866e-17  2.29344373e-17 -3.14634869e-17  1.77464674e-17
 -7.06340691e-17  3.53487526e-17  4.97225705e-17  3.19098753e-17
  4.39477364e-17 -1.04224560e-16  2.68682370e-17  2.41683582e-17
  1.35278585e-16 -3.12548535e-19  3.18576437e-17  7.81686152e-17
  9.36135027e-17 -1.69277333e-16 -6.42521684e-17  1.79731518e-16
 -8.64463942e-17 -5.81525098e-17 -5.61344509e-17 -5.67538960e-17
 -4.91026453e-17 -6.18871237e-17  2.44923557e-17  8.80679480e-17]
```

```

3.39407396e-17 -7.99458932e-17 -1.83359801e-17 -4.88855638e-17
-6.26084410e-17 8.59936322e-17 8.93320725e-17 9.20231139e-17
1.61506588e-17 -1.78465435e-16 -3.22325999e-17 9.46469086e-18
1.48693597e-17 -4.20781534e-17 -8.42958080e-17 5.98917242e-17
-1.54771432e-17 1.28122649e-17 -3.87064095e-17 1.76169566e-16
-1.19933057e-17 -1.56504807e-16 1.03941723e-16 -1.24932737e-17
-2.23403305e-17 -3.46406249e-17 1.01816738e-16 -1.84545951e-17
-4.63733706e-17 -4.23510389e-17 -8.05495242e-17 1.04224560e-16
2.90054998e-17 -1.37271947e-17 -1.89082402e-16 -1.65816089e-17
-6.74975227e-19 -6.12744577e-17 -9.02643028e-17 1.58836169e-16
2.76503806e-17 -1.79731518e-16 1.42320131e-16 6.85936733e-17
2.33063347e-18 3.98597385e-17 8.02853138e-17 7.87812811e-17
-2.11431558e-17 -9.85091116e-17 -7.05425274e-17 7.99458932e-17
7.42097169e-17 5.93267273e-17 8.80462364e-18 -1.02887790e-16
-5.81494040e-17 -7.51289564e-17 -1.28014589e-17 1.68024271e-16]

```

7.2 Net error

The sum of absolute value of the error gives the net error associated with the IDFT of the above signal. The net error is rather small so the reconstruction is reliable.

```
[6]: print(np.sum(np.abs(np.imag(np.fft.ifft(fft)))))
```

```
6.351558611224217e-15
```

8 Extending the understanding for 2D Fourier Transform

As mentioned earlier the 1D DFT is derived from the 1D Continuous Fourier Transform, the 2D DFT can also be derived from 2D Continuous Fourier Transform. The derivations are avoided here. Since Image is a 2D array of values, the 2D DFT is required to take the transform of it.

The 2D DFT is defined as,

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y) e^{-2\pi i (ux/M + vy/N)} \quad (15)$$

Here u = frequency in x direction, v = frequency in y direction, M = No. of rows, N = No. of cols. x and y = spatial variable (pixels for images)

For Images, the 2D DFT is nothing but taking a 1D DFT of the rows and then taking the 1D DFT of the cols or vice versa. This is why the 1D DFT insights hold in the 2D DFT also.

The 2D IDFT is defined as,

$$f(x, y) = \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v) e^{2\pi i (ux/M + vy/N)} \quad (16)$$

8.1 Simple sine wave

For Illustration of how the sine waves react when 2D DFT is computed, a sine wave along the x axis has been taken that is constant along the y axis. The signal $f(t) = \sin(2\pi x)$ has been taken.

```
[7]: import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

x = np.linspace(0, 1.0, 100)
y = np.linspace(0, 1.0, 100)
# Constructing a grid from the x and y arrays
x, y = np.meshgrid(x, y)

freq = 2
z = np.sin(2 * np.pi * freq * x) # sine wave along x, constant along y

# plot 3D
fig = plt.figure(figsize=(13, 10))

# Plot 1
ax1 = fig.add_subplot(221, projection='3d')

ax1.plot_surface(x, y, z, cmap='magma')
ax1.set_title("3D Sin(2pi2x)")
ax1.set_xlabel('Frequencies X axis')
ax1.set_ylabel('Frequencies Y axis')
ax1.set_zlabel('Z Axis')

#plot 2
ax2 = fig.add_subplot(222, projection='3d')

ax2.plot_surface(x, y, z, cmap='magma')

ax2.set_xlabel('Frequencies X axis')
ax2.set_zlabel('Z Axis')
ax2.set_zticks([])
ax2.set_yticks([])
ax2.set_title("viewing angle normal to xz plane")

ax2.view_init(elev=0, azim=90)

# plot 3
```

```

ax3 = fig.add_subplot(223, projection='3d')
ax3.plot_surface(x, y, z, cmap='magma')

ax3.set_ylabel('Frequencies Y axis')
ax3.set_zlabel('Z Axis')
ax3.set_zticks([])
ax3.set_xticks([])
ax3.set_title("viewing angle normal to yz plane")

ax3.view_init(elev=0, azim=180)

# plot 4
ax4 = fig.add_subplot(224, projection='3d')
ax4.plot_surface(x, y, z, cmap='gray')

ax4.set_ylabel('Frequencies Y axis')
ax4.set_xlabel('Frequencies X axis')

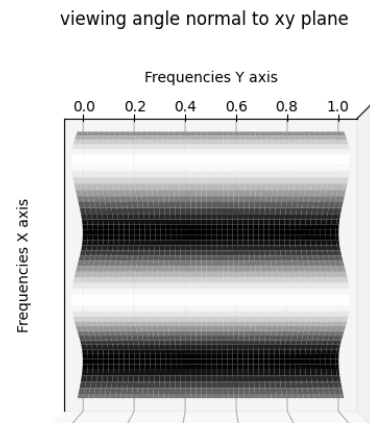
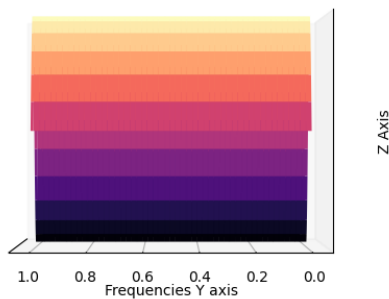
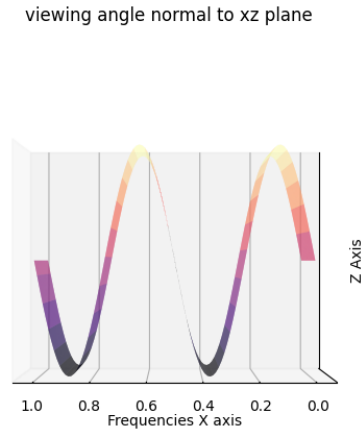
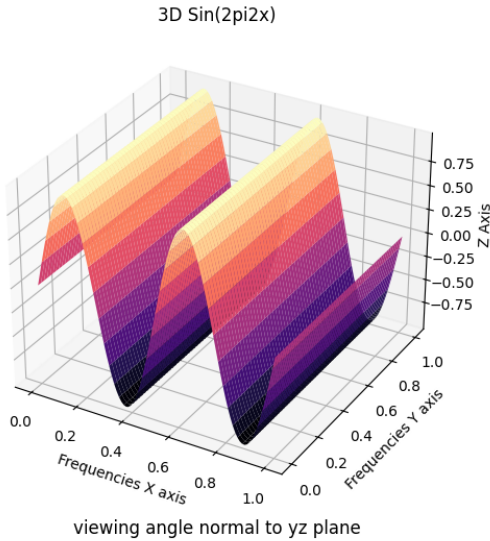
ax4.set_zticks([])
ax4.set_xticks([])
ax4.set_title("viewing angle normal to xy plane")

ax4.view_init(elev=90, azim=0)

# Plot adjustments
plt.subplots_adjust(wspace=-0.3, hspace = -0.5)

plt.tight_layout()
plt.show()

```



The plot in top left is the 3D plot of $\sin(2\pi 2x)$, top right of that is how the graph looks in the x direction which is a sine wave, bottom left is how the graph looks in the y direction which is constant, bottom right is how the graph looks from the top. The graph viewed from the top is very important and it is an image of normal sine function of frequency 2. It is a common practice to take the DFT of such simple images to try and understand what the DFT is doing to an image.

It is to be noted that because the 3D graph is viewed from above, the shape looks a little wavy, but in an actual 2D image the same pattern will be seen without the wavy shapes parallel to x axis.

Now the DFT of the 3D signal is taken.

```
[8]: dft = cv.dft(np.float32(z), flags = cv.DFT_COMPLEX_OUTPUT)

dft = np.fft.fftshift(dft)

magnitude, phase = cv.cartToPolar(dft[:, :, 0], dft[:, :, 1])
```

```

fig = plt.figure(figsize=(13, 11))
ax1 = fig.add_subplot(221, projection='3d')

n = len(magnitude)

X = np.linspace(-n//2, n//2, n)
Y = np.linspace(-n//2, n//2, n)
X, Y = np.meshgrid(X,Y)

ax1.plot_surface(X, Y, magnitude)

ax1.set_xlabel('Frequencies X axis')
ax1.set_ylabel('Frequencies Y axis')
ax1.set_zlabel('Amplitude')
ax1.set_title("3D Sin(2pi2x) Magnitude Spectrum")

ax2 = fig.add_subplot(222, projection='3d')

ax2.plot_surface(X, Y, magnitude)

ax2.set_xlabel('Frequencies X axis')
ax2.set_zlabel('Amplitude')
ax2.set_zticks([])
ax2.set_yticks([])
ax2.set_title("Magnitude Spectrum view orthogonal to xz plane")

ax2.view_init(elev=0, azim=90)

# plot 3
ax3 = fig.add_subplot(223, projection='3d')
ax3.plot_surface(X, Y, magnitude)

ax3.set_ylabel('Frequencies Y axis')
ax3.set_zlabel('Amplitude')
ax3.set_zticks([])
ax3.set_xticks([])
ax3.set_title("Magnitude Spectrum view orthogonal to yz plane")

ax3.view_init(elev=0, azim=180)

# plot 4
ax4 = fig.add_subplot(224, projection='3d')
ax4.plot_surface(X, Y, phase)

```

```

ax4.set_ylabel('Frequencies Y axis')
ax4.set_xlabel('Frequencies X axis')

ax4.set_zticks([])
ax4.set_xticks([])
ax4.set_title("Phase Spectrum")

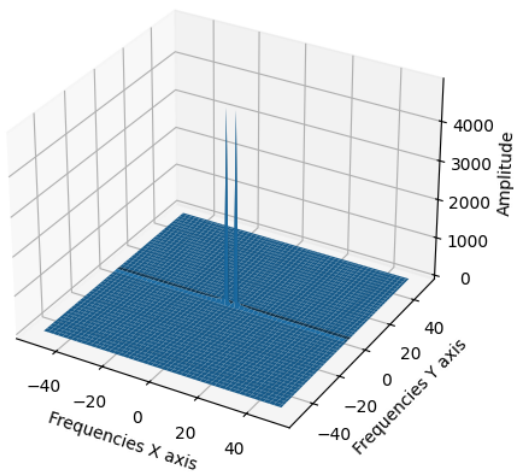
# ax4.view_init(elev=90, azim=0)

plt.subplots_adjust(wspace=-0.0, hspace = 0.1)

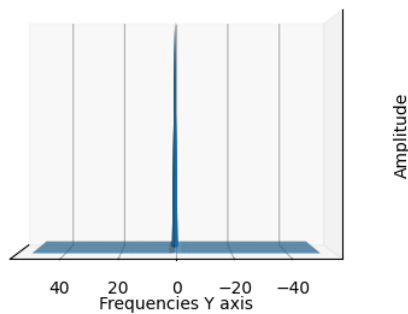
# ax.view_init(elev=0, azim=180)
# Show the plot
plt.show()

```

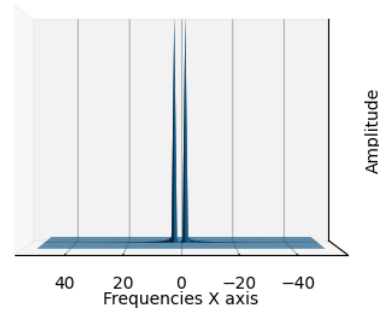
3D Sin(2pi2x) Magnitude Spectrum



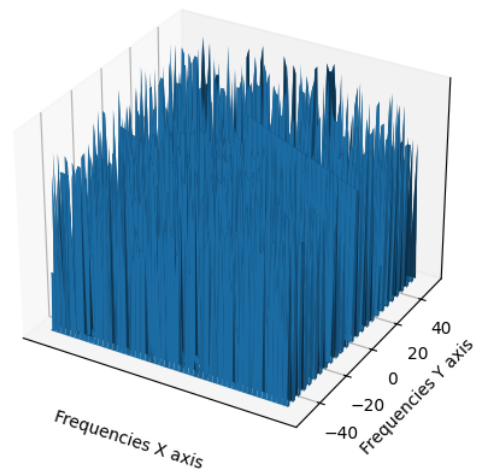
Magnitude Spectrum view orthogonal to yz plane



Magnitude Spectrum view orthogonal to xz plane



Phase Spectrum



Now the above results are consistent with the previous discussion. As discussed, 2D DFT is similar to taking two 1D DFT. One DFT is taken along the x axis and the other one is taken in the y direction. In the x axis it has already been shown that the signal is a sine wave with frequency 2. So in the x axis in the magnitude spectrum two spikes are seen at frequency 2. This finding is consistent with the 1D cases discussed so far. In the y axis as discussed above there is a constant function and the constant function is equivalent to an impulse at the origin (can be said from proof in appendix) which is what is seen in the amplitude spectrum when viewed from the y axis.

The phase spectrum is also like the 1D case, quite random and unpredictable.

8.2 Sum of sines

For this section the signal has been created by summing two same sine functions one along the x axis and the other along the y axis, to see how then react to 2D DFT. The Sine functions that are summed in this section is, $\sin(2\pi 3x)$ and $\sin(2\pi 3y)$.

```
[9]: import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D

x = np.linspace(0, 1.0, 50)
y = np.linspace(0, 1.0, 50)
# Constructing a grid from the x and y arrays
x, y = np.meshgrid(x, y)

freq = 3
z = np.sin(2 * np.pi * freq * x) + np.sin(2 * np.pi * freq * y) # sine wave
    ↪ along x, constant along y

# plot 3D
fig = plt.figure(figsize=(13, 10))

# Plot 1
ax1 = fig.add_subplot(221, projection='3d')

ax1.plot_surface(x, y, z, cmap='magma')

ax1.set_xlabel('Frequencies X axis')
ax1.set_ylabel('Frequencies Y axis')
ax1.set_zlabel('Z Axis')
ax1.set_title("3D  $\sin(2\pi 3x) + \sin(2\pi 3y)$ ")

#plot 2
```

```

ax2 = fig.add_subplot(222, projection='3d')

ax2.plot_surface(x, y, z, cmap='magma')

ax2.set_xlabel('Frequencies X axis')
ax2.set_zlabel('Z Axis')
ax2.set_zticks([])
ax2.set_yticks([])
ax2.set_title("View orthogonal to xz")

ax2.view_init(elev=0, azim=90)

# plot 3
ax3 = fig.add_subplot(223, projection='3d')
ax3.plot_surface(x, y, z, cmap='magma')

ax3.set_ylabel('Frequencies Y axis')
ax3.set_zlabel('Z Axis')
ax3.set_zticks([])
ax3.set_xticks([])
ax3.set_title("View orthogonal to yz")

ax3.view_init(elev=0, azim=180)

# plot 4
ax4 = fig.add_subplot(224, projection='3d')
ax4.plot_surface(x, y, z, cmap='gray')

ax4.set_ylabel('Frequencies Y axis')
ax4.set_xlabel('Frequencies X axis')

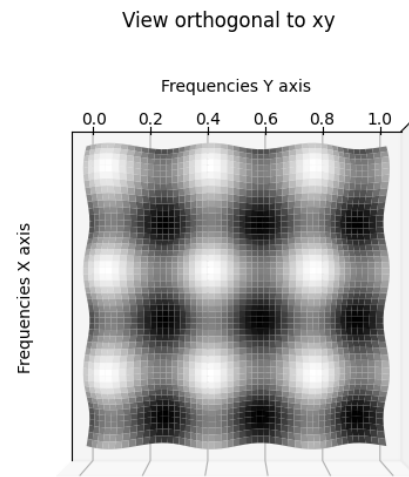
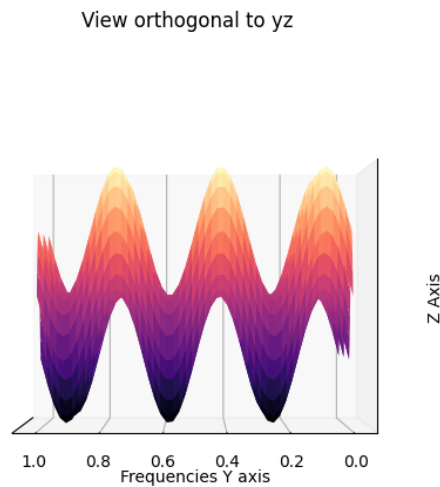
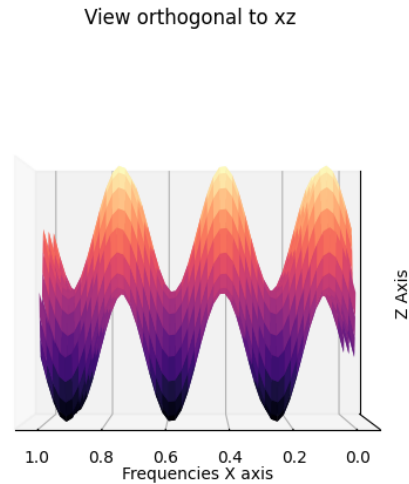
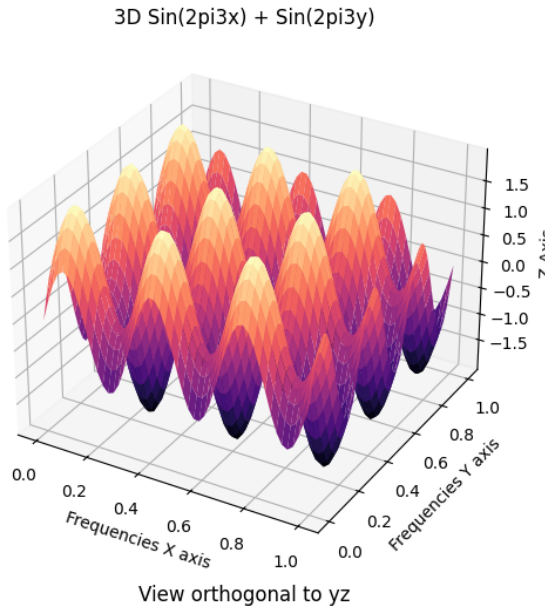
ax4.set_zticks([])
ax4.set_xticks([])
ax4.set_title("View orthogonal to xy")

ax4.view_init(elev=90, azim=0)

# Plot adjustments
plt.subplots_adjust(wspace=-0.3, hspace = 0.8)

plt.tight_layout()
plt.show()

```



The graph on top left is the 3D function made of $\sin(2\pi 3x) + \sin(2\pi 3y)$, the one on top right is the view of the graph orthogonal to xz plane that shows a sine function with frequency 3 along the x axis, the bottom left is the view of the graph orthogonal to yz plane that shows a sine function with frequency 3 along the y axis, the one on the bottom right is the top view of the 3D function. The top view is an image of sum of sines of frequency 3. The wavy pattern is due to the 3D nature of the graph, but if one sees some online pictures of sum of sines of same frequency the images patterns will be the same as the bottom right image.

```
[10]: #Computing the DFT
dft = cv.dft(np.float32(z), flags = cv.DFT_COMPLEX_OUTPUT)
dft = np.fft.fftshift(dft)
```

```

# Computing magnitude and phase
magnitude, phase = cv.cartToPolar(dft[:, :, 0], dft[:, :, 1])

fig = plt.figure(figsize=(13, 11))
ax1 = fig.add_subplot(121, projection='3d')

n = len(magnitude)

X = np.linspace(-n//2, n//2, n)
Y = np.linspace(-n//2, n//2, n)
X, Y = np.meshgrid(X, Y)

ax1.plot_surface(X, Y, magnitude)

# Set labels
ax1.set_xlabel('Frequencies along X axis')
ax1.set_ylabel('Frequencies along Y axis')
ax1.set_zlabel('Amplitude')
ax1.set_title("3D Sin(2pi3x) + Sin(2pi3y) amplitude spectrum")

# plot 4
ax2 = fig.add_subplot(122, projection='3d')
ax2.plot_surface(X, Y, phase)

ax2.set_ylabel('Frequencies along Y axis')
ax2.set_xlabel('Frequencies along X axis')

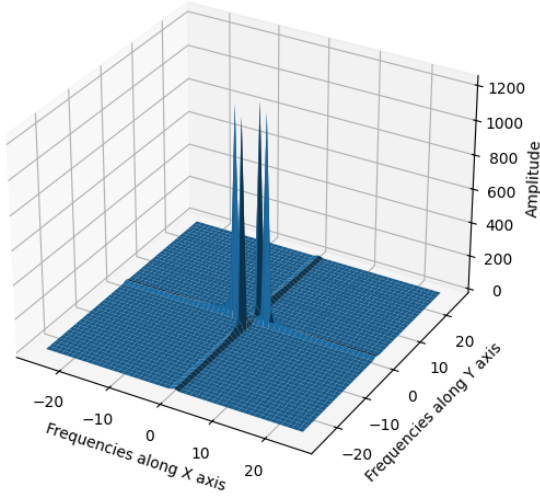
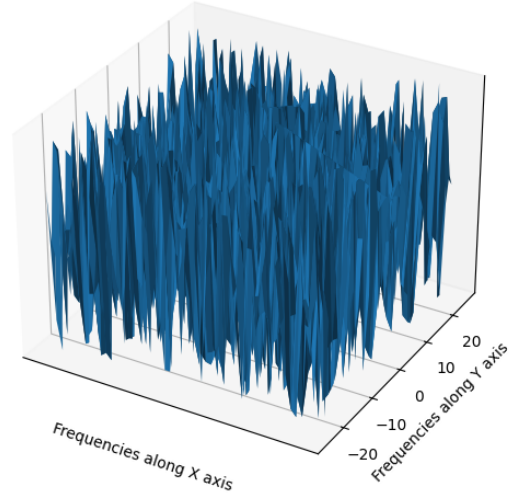
ax2.set_zticks([])
ax2.set_xticks([])
ax2.set_title("3D Sin(2pi3x) + Sin(2pi3y) phase spectrum")

# ax4.view_init(elev=90, azimuth=0)

plt.subplots_adjust(wspace=0.1, hspace = -0.0)

# ax.view_init(elev=0, azimuth=180)
# Show the plot
plt.show()

```

3D $\sin(2\pi 3x) + \sin(2\pi 3y)$ amplitude spectrum3D $\sin(2\pi 3x) + \sin(2\pi 3y)$ phase spectrum

In this case the results are also completely consistent with the prior findings. In the original image it has been shown that the image has sine function of frequency 3 in both the x and y directions. So it is expected that there will be two impulses in each x and y direction at frequency value 3. This expected outcome is what is visible in the image on the left. The random looking phase information is on the right.

8.3 Observations based on experiments

From the analysis so far it can be concluded that taking DFT of a signal reveals the contribution of each frequencies and the phase information, of the underlying approximating function. The frequencies that are present in the underlying approximating function, have spikes.

It is to be noted that if the amplitude spectrum is seen from the top which is the image representation of the amplitude spectrum, it will look like some dots where the number of impulses will be equal to the number of dots and the values of frequencies present determine how closely of how far the dots are situated.

Another observation is, in all the magnitude spectrums that were displayed here, the impulses sit on top of the axis created by the nyquist frequencies. But it should not be mistaken that the impulses(white dots in 2D amplitude spectrum image) will always be on top of the axis. The impulses are on top of the axis as all of the Sine function taken into consideration were parallel to the axis. If the sine function considered were rotated, say parallel with a line with slope 1, it would be possible to see impulses(white dots in 2D amplitude spectrum image) that are at an angle in the amplitude spectrum.

9 Fourier Transform of a real image

Now DFT's of real images need to be considered. The images from now on will be strictly 2D and they are rather confusing to interpret. But the observations from the previous experiments will help gain some insight into these images. It is to be noted, the amplitude spectrum in 2D

images will look like some white dots symmetric about the origin. These dots give the information that the amplitude at those frequencies are having a contribution on the underlying approximating function. The brighter the dots the more the contribution. The question then is what purpose does this information serve? One of the example would be in an image, if all the white dots are around the center, and there are only few outliers that are very far from the concentrated white dot area, then the chances are these outliers are nothing but noise. So if these outliers are removed and the image is reconstructed using IDFT, then a better version of the image might appear.

Apart from amplitude spectrum of an image the phase spectrum is also important. The phase information represents the edge or boundary information of images.

The Nyquist frequencies in x axis and y axis divide the amplitude spectrum in four quadrants. Quadrant 1 and 3 are mirror images of each other. Quadrant 2 and 4 are mirrors of each other.

9.1 Fourier Transform of image 1

```
[11]: import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread("G:
↳\\Fourier-report\\Selected_images\\COD10K-CAM-1-Aquatic-3-Crab-47.jpg", cv.
↳IMREAD_GRAYSCALE)
print(img.shape)

dft = np.fft.fft2(img)

dft = np.fft.fftshift(dft)

magnitude_original_img1 = np.abs(dft)
phase_img1 = np.angle(dft)

magnitude_img1 = 20*np.log(magnitude_original_img1)

combined0 = np.multiply(magnitude_original_img1, np.exp(1j* phase_img1))

combined0 = np.fft.ifftshift(combined0)
reconstructed_image = np.fft.ifft2(combined0)
reconstructed_image = np.abs(reconstructed_image)

# Plotting
fig, ax = plt.subplots(2,2)

fig.set_size_inches(12, 8)
# magnitude = cv.cvtColor(magnitude, cv.COLOR_GRAY2GRAY)
```

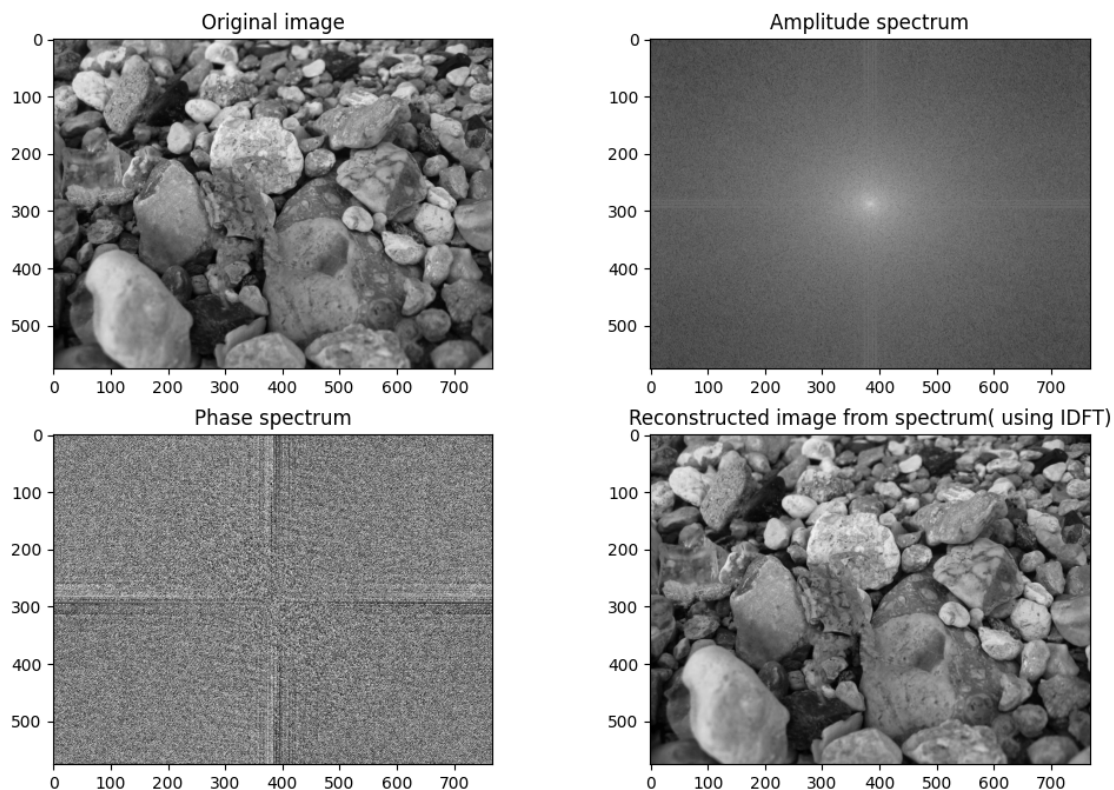
```

ax[0,0].imshow(img, cmap = 'gray')
ax[0,0].set_title("Original image")
ax[0,1].imshow(magnitude_img1, cmap = 'gray')
ax[0,1].set_title("Amplitude spectrum")
ax[1,0].imshow(phase_img1, cmap = 'gray')
ax[1,0].set_title("Phase spectrum")
ax[1,1].imshow(reconstructed_image, cmap = 'gray')
ax[1,1].set_title("Reconstructed image from spectrum( using IDFT)")

# plt.adjust_subplots()
plt.show()

```

(576, 768)



From the above image it is visible that the amplitude spectrum has heavily concentrated white dots at the center. This means the underlying approximating function made up of sum of sines and cosines, that approximate the image, have very strong low frequency components and very weak high frequency components. In images of such sort, if compression is required to reduce size, it is possible to remove some of high frequency components and still have a good reconstruction of the original image.

9.2 Fourier Transform of image 2

```
[12]: import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread("G:
↪\\Fourier-report\\Selected_images\\COD10K-CAM-1-Aquatic-13-Pipefish-595.
↪jpg", cv.IMREAD_GRAYSCALE)
img = cv.resize(img, (768,576))
print(img.shape)

dft = np.fft.fft2(img)

dft = np.fft.fftshift(dft)

magnitude_original_img2 = np.abs(dft)
phase_img2 = np.angle(dft)

magnitude_img2 = 20*np.log(magnitude_original_img2)

combined1 = np.multiply(magnitude_original_img2, np.exp(1j* phase_img2))

alpha = np.fft.ifftshift(combined1)
reconstructed_image = np.fft.ifft2(alpha)
reconstructed_image = np.abs(reconstructed_image)

# Plotting
fig, ax = plt.subplots(2,2)

fig.set_size_inches(12, 8)
# magnitude = cv.cvtColor(magnitude, cv.COLOR_GRAY2GRAY)
ax[0,0].imshow(img, cmap = 'gray')
ax[0,0].set_title("Original image")
ax[0,1].imshow(magnitude_img2, cmap = 'gray')
ax[0,1].set_title("Amplitude spectrum")

ax[1,0].imshow(phase_img2, cmap = 'gray')
ax[1,0].set_title("Phase spectrum")

ax[1,1].imshow(reconstructed_image, cmap = 'gray')
ax[1,1].set_title("Reconstructed image from spectrum( using IDFT)")
```

```
plt.show()
```

(576, 768)

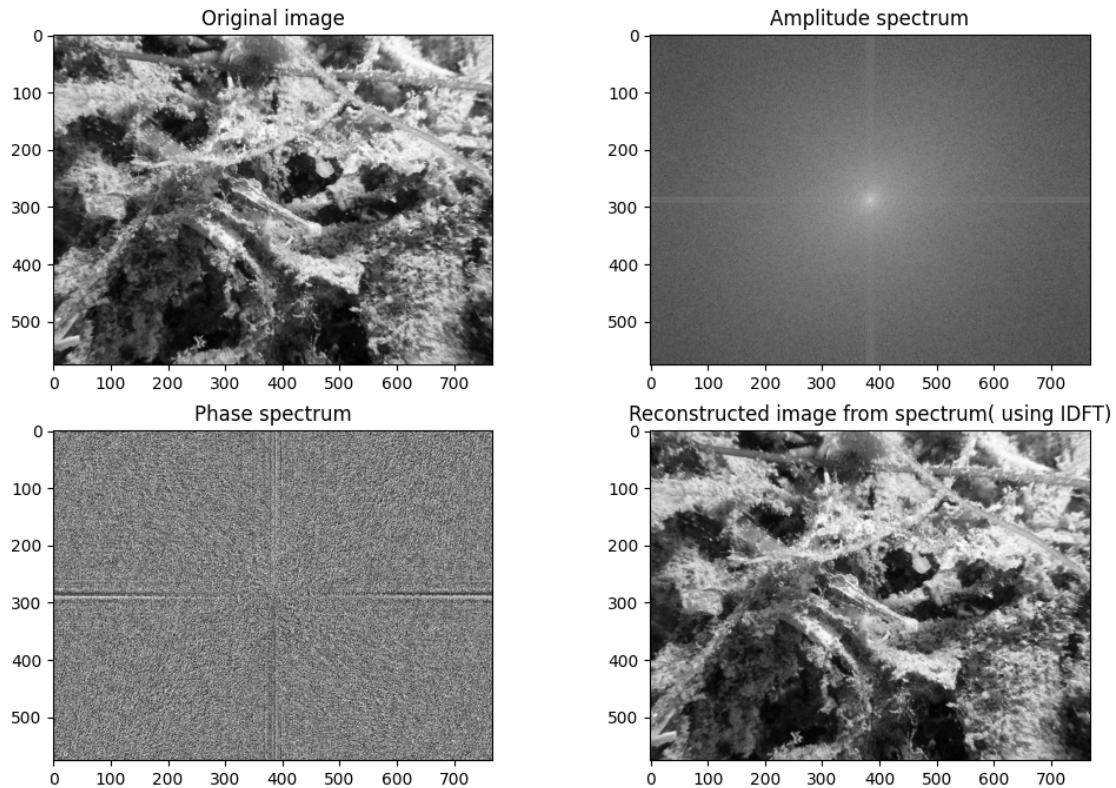


Image 2 has similar interpretations as image 1

9.3 Inverse Fourier Transform with swapped spectral information

In this section the image spectral information are swapped with each other to understand what difference does it make. Here firstly the both images have been converted to the same size. Then the DFT is done to separate the magnitude and phase of each image. Then Magnitude of image1 and phase of image 2 is combined to create an image and amplitude of image2 and phase of image 1 is combined to create another image.

```
[13]: # prep = cv.polarToCart(magnitude_original_img1, phase_img2)

combined2 = np.multiply(magnitude_original_img1, np.exp(1j* phase_img2))
combined3 = np.multiply(magnitude_original_img2, np.exp(1j* phase_img1))
```

```

combined2 = np.fft.ifftshift(combined2)
reconstructed_image_alpha = np.fft.ifft2(combined2)
reconstructed_image_alpha = np.abs(reconstructed_image_alpha)

combined3 = np.fft.ifftshift(combined3)
reconstructed_image_gamma = np.fft.ifft2(combined3)
reconstructed_image_gamma = np.abs(reconstructed_image_gamma)

# Plotting
fig, ax = plt.subplots(2,2)

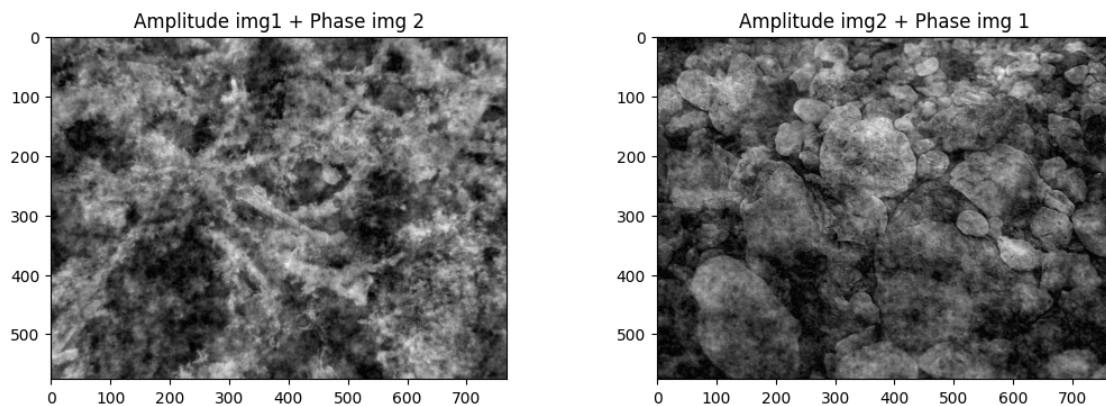
fig.set_size_inches(11, 7)
# magnitude = cv.cvtColor(magnitude, cv.COLOR_GRAY2GRAY)
ax[0,0].imshow(reconstructed_image_alpha, cmap = 'gray')
ax[0,0].set_title("Amplitude img1 + Phase img 2")

ax[0,1].imshow(reconstructed_image_gamma, cmap = 'gray')
ax[0,1].set_title("Amplitude img2 + Phase img 1")

ax[1,0].remove()
ax[1,1].remove()

plt.tight_layout()
plt.show()

```



After swapping the spectral information both images are recognizable but look blurry. It is clear that some information has been lost in both the images.

10 Reconstructing image only from amplitude or phase

The Amplitude only reconstruction = Amplitude of image * e^{j*0} , here phase = 0

The phase only reconstruction = $1 * e^{j*\phi}$, here amplitude = 1 and ϕ = phase of image

```
[14]: import numpy as np
import cv2 as cv
from matplotlib import pyplot as plt

img = cv.imread("G:
↪\\Fourier-report\\Selected_images\\COD10K-CAM-1-Aquatic-13-Pipefish-595.
↪jpg", cv.IMREAD_GRAYSCALE)
img = cv.resize(img, (768,576))

dft = np.fft.fft2(img)

dft = np.fft.fftshift(dft)

magnitude_original_img2 = np.abs(dft)
phase_img2 = np.angle(dft)

magnitude_only = np.multiply(magnitude_original_img2, np.exp(1j* 0))
phase_only = np.multiply(1, np.exp(1j* phase_img2))

magnitude_only = 20*np.log(magnitude_only)
magnitude_only_reconstruction = np.fft.ifftshift(magnitude_only)
magnitude_only_reconstruction = np.fft.ifft2(magnitude_only_reconstruction)
magnitude_only_reconstruction = np.abs(magnitude_only_reconstruction)

phase_only_reconstruction = np.fft.ifftshift(phase_only)
phase_only_reconstruction = np.fft.ifft2(phase_only_reconstruction)
phase_only_reconstruction = np.abs(phase_only_reconstruction)

# Plotting
fig, ax = plt.subplots(2,2)

fig.set_size_inches(12, 8)

ax[0,0].imshow(img, cmap = 'gray')
ax[0,0].set_title("Original image")

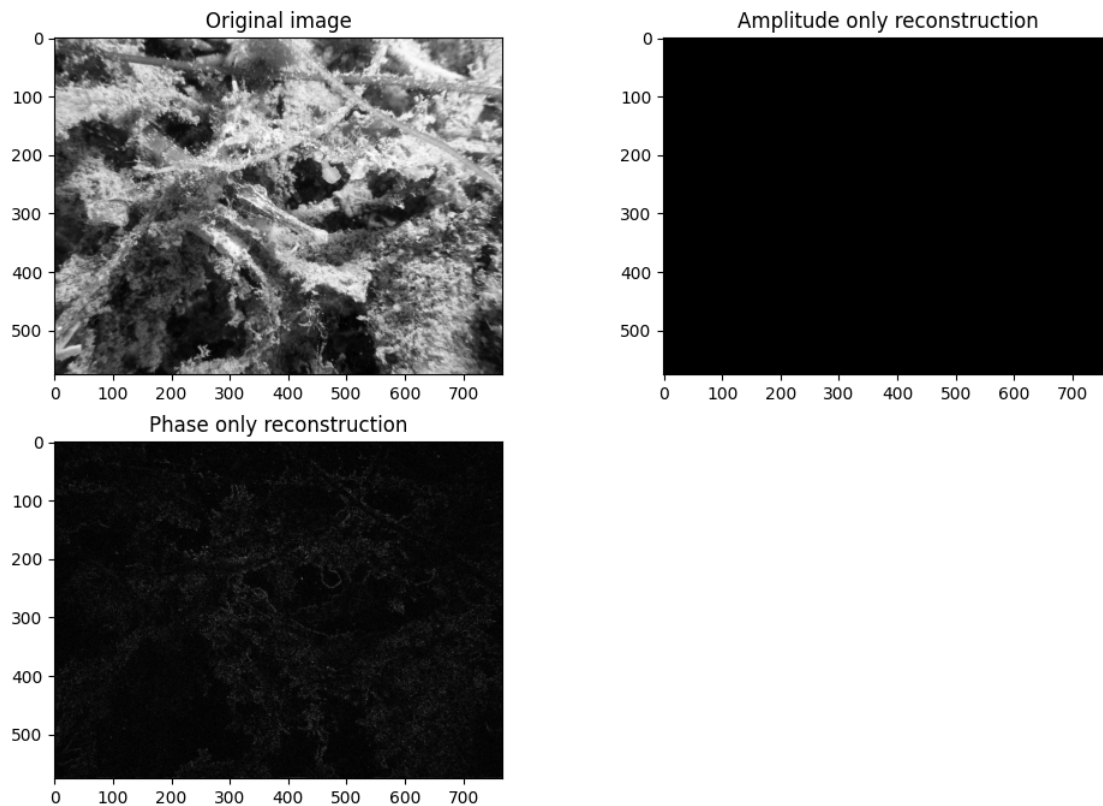
ax[0,1].imshow(magnitude_only_reconstruction, cmap = 'gray')
ax[0,1].set_title("Amplitude only reconstruction")

ax[1,0].imshow(phase_only_reconstruction, cmap = 'gray')
```

```
ax[1,0].set_title("Phase only reconstruction")

ax[1,1].remove()

# # plt.adjust_subplots()
# plt.show()
```



The Amplitude only reconstruction is totally black (even after log transform), but the phase only reconstruction has very faint lines. These lines are actually edges of the original image. This result of the phase image is expected as the phase represents the edge or boundary information of an image.

11 References

1. <https://www.youtube.com/watch?v=gZNm7L96pfY&list=PLB24BC7956EE040CD> (Lecture 1-6)
2. Prof. Brad Osgood, The Fourier Transform and its Applications, Lecture notes, pp 1-21, 65-75
3. R.C. Gonzalez, R.E Woods, Digital Image Processing, Third Edition, Pearson, 2008, pp. 202-213, 220-226, 235-236

4. S.W. Smith, The Scientist and Engineer's Guide to Digital Signal Processing, chapter 24: Fourier Image Analysis (online)
5. C. Solomon, T. Breckon, Fundamentals of Digital Image Processing, Wiley, Global edition, pp 115-126
6. M. Schwartz, Fourier transforms, pp 1-6
7. CS1114 Section 8: The Fourier Transform
8. I.T. Young, J.J. Gerbrands, L.J.v. Vliet, Fundamentals of Image Processing
9. Math 563 Lecture Notes The discrete Fourier transform

12 Appendix

12.0.1 Proof :

Proof of the statement that $\int_0^1 e^{2\pi i(1-u)t} dt = \delta(u-1)$ and $\int_0^1 e^{-2\pi i(1-u)t} dt = \delta(u+1)$ where $\delta(u-x)$ is a dirac delta function and $u \in \mathbb{Z}$

The integral needs to be evaluated,

$$\int_0^1 e^{2\pi i(1-u)t} dt = \frac{1}{2\pi i(1-u)} e^{2\pi i(1-u)t} \Big|_0^1 = \frac{1}{2\pi i(1-u)} (e^{2\pi i(1-u)} - e^0) \quad (17)$$

$$\int_0^1 e^{2\pi i(1-u)t} dt = \frac{1}{2\pi i(1-u)} (e^{2\pi i(1-u)} - 1) \quad (18)$$

In the above integral there can be two cases, one arises if $u = 1$ and the other when $u \neq 1$

Here if the value $u \neq 1$ then, $1-u \in \mathbb{Z}^+ \text{ Or } \mathbb{Z}^-$,

$$e^{2\pi i(1-u)} = \cos(2\pi(1-u)) + i\sin(2\pi(1-u)) \quad (19)$$

$$\cos(2\pi(\text{integer})) = \cos(2\pi) = 1, \sin(2\pi(\text{integer})) = \sin(2\pi) = 0 \quad (20)$$

so,

$$\int_0^1 e^{2\pi i(1-u)t} dt = \frac{1}{2\pi i(1-u)} (1 - 1) = 0 \quad (21)$$

And if $u = 1$, then $1-u = 0$,

$$\int_0^1 e^{2\pi i(1-u)t} dt = \int_0^1 e^0 dt = 1 \quad (22)$$

The summary of the above two cases can be stated the following,

$$\int_0^1 e^{2\pi i(1-u)t} dt = \begin{cases} 1 & \text{if } u = 1 \\ 0 & \text{if } u \neq 1 \end{cases} \quad (23)$$

This is equivalent to,

$$\delta(u-1) = \begin{cases} 1 & \text{if } u = 1 \\ 0 & \text{if } u \neq 1 \end{cases} \quad (24)$$

So,

$$\int_0^1 e^{2\pi i(1-u)t} dt = \delta(u-1) \quad (25)$$

Similarly it can be proven,

$$\int_0^1 e^{-2\pi i(1-u)t} dt = \delta(u+1) \quad (26)$$

[]: